어텐션 메커니즘 상세 분석 (Attention Mechanism)

목차

- 개요
- 어텐션의 직관적 이해
- 스케일드 닷-프로덕트 어텐션
- Query, Key, Value 개념
- 수학적 공식 상세 분석
- PyTorch 구현
- 어텐션 가중치 시각화
- 실습 예제
- 핵심 개념 정리

개요

어텐션 메커니즘(Attention Mechanism)은 트랜스포머 아키텍처의 핵심이자 혁신적인 요소입니다. 기존의 RNN이나 LSTM에서 순차적으로 정보를 처리하던 방식과 달리, 어텐션은 입력 시퀀스의 모든 위치에 동시에 접근하여 중요한 정보에 집중할 수 있게 해줍니다.

어텐션이 해결하는 문제

1. **장거리 의존성 문제**: RNN에서 멀리 떨어진 단어들 간의 관계를 학습하기 어려운 문제
2. **병목 현상**: 인코더의 마지막 은닉 상태에 모든 정보를 압축해야 하는 문제

3. **병렬화 불가**: 순차적 처리로 인한 계산 효율성 문제

어텐션의 직관적 이해

어텐션을 일상적인 예시로 이해해보겠습니다:

번역 예시: “The cat sat on the mat” → “고양이가 매트 위에 앉았다”

"앉았다"를 번역할 때, 모델은:

- "sat"에 가장 높은 주의(attention)를 기울임
- "cat"과 "mat"에도 어느 정도 주의를 기울임 (주어와 목적어 관계)
- “the”, "on"에는 상대적으로 낮은 주의를 기울임

이처럼 어텐션은 **현재 처리하고 있는 단어와 관련된 다른 단어들에 얼마나 집중할지**를 학습합니다.

스케일드 닷-프로덕트 어텐션

트랜스포머에서 사용하는 어텐션은 "Scaled Dot-Product Attention"입니다.

기본 공식

```
Attention(Q, K, V) = softmax(QK^T / √d_k)V
```

여기서:

- Q (Query): 질의 행렬
- K (Key): 키 행렬
- V (Value): 값 행렬
- d_k: 키 벡터의 차원
- √d_k: 스케일링 팩터

단계별 계산 과정

1. **유사도 계산**: Q와 K의 내적으로 유사도 점수 계산
2. **스케일링**: √d_k로 나누어 그래디언트 안정화
3. **정규화**: softmax로 확률 분포 변환
4. **가중합**: 어텐션 가중치와 V의 가중합 계산

Query, Key, Value 개념

어텐션 메커니즘의 핵심은 Query, Key, Value 개념입니다. 이는 정보 검색 시스템에서 영감을 받았습니다.

정보 검색 시스템 비유

도서관에서 책을 찾는 상황:

- **Query (질의)**: "머신러닝에 관한 책을 찾고 싶다"
- **Key (키)**: 각 책의 제목, 키워드, 카테고리
- **Value (값)**: 실제 책의 내용

도서관 시스템은:

1. 질의와 각 책의 키를 비교하여 관련도 계산
2. 관련도가 높은 책들을 우선적으로 선택
3. 선택된 책들의 내용(값)을 가중평균으로 조합

트랜스포머에서의 Q, K, V

셀프 어텐션의 경우:

- 입력 시퀀스의 각 위치가 동시에 Query, Key, Value 역할
- 각 위치에서 다른 모든 위치와의 관련성 계산

예시: "The cat sat on the mat"에서 “cat” 위치의 관점

```
Query: "cat"의 표현 벡터

Keys: ["The", "cat", "sat", "on", "the", "mat"]의 표현 벡터들

Values: ["The", "cat", "sat", "on", "the", "mat"]의 표현 벡터들



결과: "cat"과 관련성이 높은 단어들("sat", "mat")에 더 높은 가중치
```

선형 변환을 통한 Q, K, V 생성

실제로는 입력 벡터를 서로 다른 선형 변환으로 Q, K, V를 생성합니다:

```
Q = XW_Q (Query 생성)

K = XW_K (Key 생성)

V = XW_V (Value 생성)
```

여기서:

- X: 입력 행렬 (seq_len × d_model)
- W_Q, W_K, W_V: 학습 가능한 가중치 행렬들

이렇게 서로 다른 변환을 사용하는 이유:

1. **역할 분리**: Query는 “무엇을 찾을지”, Key는 “무엇과 매칭할지”, Value는 “실제 정보”
2. **표현력 향상**: 같은 입력에서 다양한 관점의 표현 생성
3. **학습 효율성**: 각각 다른 패턴을 학습할 수 있음

수학적 공식 상세 분석

1. 어텐션 점수 계산

```
scores = QK^T / √d_k
```

차원 분석:

- Q: (seq_len × d_k)
- K^T: (d_k × seq_len)
- QK^T: (seq_len × seq_len)

결과 행렬의 (i,j) 원소는 i번째 Query와 j번째 Key의 유사도를 나타냅니다.

2. 스케일링의 필요성

√d_k로 나누는 이유:

문제: d_k가 클 때, 내적 값이 매우 커져서 softmax가 포화상태가 됨

```
예시: d_k=512일 때, 내적 값이 수백~수천이 될 수 있음
softmax(1000) ≈ 1, softmax(-1000) ≈ 0 (극단적 확률)
```

해결: √d_k로 나누어 분산을 1로 정규화

```
Var(QK^T / √d_k) ≈ 1 (Q, K가 표준정규분포를 따를 때)
```

3. Softmax 정규화

```
attention_weights = softmax(scores) = exp(scores_i) / Σ_j exp(scores_j)
```

특성:

- 모든 가중치의 합이 1: Σ_i attention_weights_i = 1
- 모든 값이 0과 1 사이: 0 ≤ attention_weights_i ≤ 1
- 확률적 해석 가능: 각 위치에 대한 주의 확률

4. 가중합 계산

```
output = attention_weights × V
```

차원 분석:

- attention_weights: (seq_len × seq_len)
- V: (seq_len × d_v)
- output: (seq_len × d_v)

각 출력 벡터는 입력의 모든 위치 정보를 어텐션 가중치에 따라 조합한 결과입니다.

PyTorch 구현

기본 어텐션 함수

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import math

def  scaled_dot_product_attention(query, key, value, mask=None, dropout=None):
"""
```

```
스케일드 닷-프로덕트 어텐션 구현

Args:

query: Query 텐서 (batch_size, seq_len, d_k)

key: Key 텐서 (batch_size, seq_len, d_k)

value: Value 텐서 (batch_size, seq_len, d_v)

mask: 마스킹 텐서 (선택적)

dropout: 드롭아웃 레이어 (선택적)

Returns:

output: 어텐션 출력 (batch_size, seq_len, d_v)

attention_weights: 어텐션 가중치 (batch_size, seq_len, seq_len)

"""

d_k = query.size(-1) # 키 차원

# 1. 어텐션 점수 계산: Q × K^T / √d_k

scores = torch.matmul(query, key.transpose(-2, -1)) / math.sqrt(d_k)

# 2. 마스킹 적용 (선택적)

if mask is not None:

scores = scores.masked_fill(mask == 0, -1e9)

# 3. Softmax로 정규화

attention_weights = F.softmax(scores, dim=-1)

# 4. 드롭아웃 적용 (선택적)

if dropout is not None:

attention_weights = dropout(attention_weights)

# 5. Value와 가중합 계산
```

```
output = torch.matmul(attention_weights, value)

return output, attention_weights
```

어텐션 레이어 클래스

```
class AttentionLayer(nn.Module):

"""

완전한 어텐션 레이어 구현

"""

def  __init__(self, d_model, dropout=0.1):

super(AttentionLayer, self).__init__()

self.d_model = d_model

# Q, K, V 생성을 위한 선형 변환

self.w_q = nn.Linear(d_model, d_model)

self.w_k = nn.Linear(d_model, d_model)

self.w_v = nn.Linear(d_model, d_model)

# 출력 선형 변환

self.w_o = nn.Linear(d_model, d_model)

# 드롭아웃

self.dropout = nn.Dropout(dropout)

def  forward(self, x, mask=None):

"""

순전파
```

```
Args:

x: 입력 텐서 (batch_size, seq_len, d_model)

mask: 마스킹 텐서 (선택적)

Returns:

output: 어텐션 출력 (batch_size, seq_len, d_model)

attention_weights: 어텐션 가중치

"""

batch_size, seq_len, d_model = x.size()

# Q, K, V 생성

Q = self.w_q(x) # (batch_size, seq_len, d_model)

K = self.w_k(x) # (batch_size, seq_len, d_model)

V = self.w_v(x) # (batch_size, seq_len, d_model)

# 어텐션 계산

attention_output, attention_weights = scaled_dot_product_attention(

Q, K, V, mask, self.dropout

)

# 출력 선형 변환

output = self.w_o(attention_output)

return output, attention_weights
```

사용 예제

```
# 모델 초기화
```

```
d_model = 512

seq_len = 10

batch_size = 2


attention_layer = AttentionLayer(d_model)


# 샘플 입력 생성

x = torch.randn(batch_size, seq_len, d_model)


# 순전파

output, attention_weights = attention_layer(x)


print(f"입력 크기: {x.shape}")

print(f"출력 크기: {output.shape}")

print(f"어텐션 가중치 크기: {attention_weights.shape}")


# 어텐션 가중치 확인 (확률의 합이 1인지)

print(f"어텐션 가중치 합: {attention_weights.sum(dim=-1)[0, 0]:.4f}")

```## 어텐션

가중치 시각화

어텐션 가중치를 시각화하면 모델이 어떤 단어들에 주의를 기울이는지 직관적으로 이해할 수 있습니
```
```

시각화 코드

```python
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

def  visualize_attention(attention_weights, input_tokens, output_tokens=None,
head_idx=0, layer_name="Attention"):
"""
어텐션 가중치 히트맵 시각화
Args:
attention_weights: 어텐션 가중치 (seq_len, seq_len)
input_tokens: 입력 토큰 리스트
output_tokens: 출력 토큰 리스트 (선택적, 셀프어텐션의 경우 None)
head_idx: 헤드 인덱스 (멀티헤드의 경우)
layer_name: 레이어 이름
"""
if output_tokens is None:
output_tokens = input_tokens
# 넘파이 배열로 변환
if torch.is_tensor(attention_weights):
attention_weights = attention_weights.detach().cpu().numpy()
```

```
# 히트맵 생성
plt.figure(figsize=(10, 8))
sns.heatmap(
attention_weights,
xticklabels=input_tokens,
yticklabels=output_tokens,
cmap='Blues',
annot=True,
fmt='.3f',
cbar_kws={'label': 'Attention Weight'}
)
plt.title(f'{layer_name} - Head {head_idx}')
plt.xlabel('Input Tokens (Keys)')
plt.ylabel('Output Tokens (Queries)')
plt.xticks(rotation=45)
plt.yticks(rotation=0)
plt.tight_layout()
plt.show()


# 실제 사용 예제
def  demonstrate_attention_visualization():
"""
어텐션 시각화 데모
```

```
"""
# 샘플 문장
tokens = ["The", "cat", "sat", "on", "the", "mat", "<EOS>"]
seq_len = len(tokens)
# 어텐션 레이어 생성
attention_layer = AttentionLayer(d_model=64)
# 샘플 입력 (실제로는 임베딩 벡터)
x = torch.randn(1, seq_len, 64)
# 어텐션 계산
output, attention_weights = attention_layer(x)
# 첫 번째 배치, 첫 번째 샘플의 어텐션 가중치 시각화
attention_matrix = attention_weights[0] # (seq_len, seq_len)
visualize_attention(attention_matrix, tokens, tokens)
return attention_matrix


# 시각화 실행
attention_matrix = demonstrate_attention_visualization()
```

어텐션 패턴 분석

실제 어텐션 가중치에서 관찰할 수 있는 패턴들:

1. 대각선 패턴 (Self-Attention)

```
def analyze_diagonal_pattern(attention_weights):
```

```
"""
대각선 어텐션 패턴 분석
"""
diagonal_values = torch.diag(attention_weights)
print("대각선 어텐션 값 (자기 자신에 대한 주의):")
for i, val in  enumerate(diagonal_values):
print(f"위치 {i}: {val:.4f}")
```

2. 구문적 관계 패턴

```
def  analyze_syntactic_patterns(attention_weights, tokens):
"""
구문적 관계 어텐션 패턴 분석
"""
seq_len = len(tokens)
print("높은 어텐션 가중치를 가진 토큰 쌍:")
for i in  range(seq_len):
for j in  range(seq_len):
if attention_weights[i, j] > 0.2 and i != j: # 임계값 0.2
print(f"{tokens[i]} -> {tokens[j]}: {attention_weights[i, j]:.4f}")


# 사용 예제
tokens = ["The", "cat", "sat", "on", "the", "mat"]
```

```
# attention_weights는 위에서 계산된 결과 사용
analyze_syntactic_patterns(attention_matrix, tokens)
```

다중 헤드 어텐션 시각화

```
def visualize_multihead_attention(attention_weights, tokens, num_heads=8):
"""
멀티헤드 어텐션 시각화
Args:
attention_weights: (num_heads, seq_len, seq_len)
tokens: 토큰 리스트
num_heads: 헤드 수
"""
fig, axes = plt.subplots(2, 4, figsize=(20, 10))
axes = axes.flatten()
for head in range(num_heads):
ax = axes[head]
# 각 헤드의 어텐션 가중치
head_attention = attention_weights[head]
# 히트맵 생성
sns.heatmap(
head_attention,
xticklabels=tokens,
```

```
yticklabels=tokens,
cmap='Blues',
ax=ax,
cbar=True,
square=True
)
ax.set_title(f'Head {head + 1}')
ax.set_xlabel('Keys')
ax.set_ylabel('Queries')
plt.tight_layout()
plt.show()


# 멀티헤드 어텐션 데모
def  demo_multihead_visualization():
"""
멀티헤드 어텐션 시각화 데모
"""
tokens = ["I", "love", "machine", "learning"]
seq_len = len(tokens)
num_heads = 8
# 각 헤드별로 서로 다른 어텐션 패턴 생성 (시뮬레이션)
attention_weights = torch.zeros(num_heads, seq_len, seq_len)
for head in  range(num_heads):
```

```
# 각 헤드마다 다른 패턴 생성
if head % 2 == 0:
# 짝수 헤드: 인접한 단어에 집중
for i in  range(seq_len):
for j in  range(max(0, i-1), min(seq_len, i+2)):
attention_weights[head, i, j] = torch.rand(1) * 0.5 + 0.3
else:
# 홀수 헤드: 장거리 의존성에 집중
for i in  range(seq_len):
for j in  range(seq_len):
if  abs(i - j) > 1:
attention_weights[head, i, j] = torch.rand(1) * 0.4 + 0.2
# 각 행을 softmax로 정규화
for head in  range(num_heads):
attention_weights[head] = F.softmax(attention_weights[head], dim=-1)
visualize_multihead_attention(attention_weights, tokens, num_heads)


# 실행
demo_multihead_visualization()
```

실습 예제

예제 1: 간단한 어텐션 계산

```python
def  simple_attention_example():
"""
간단한 어텐션 계산 예제
"""
print("=== 간단한 어텐션 계산 예제 ===")
# 간단한 3개 단어 시퀀스
seq_len = 3
d_model = 4
# 입력 벡터 (단순화를 위해 작은 값 사용)
x = torch.tensor([
[1.0, 0.0, 0.0, 0.0], # "I"
[0.0, 1.0, 0.0, 0.0], # "love"
[0.0, 0.0, 1.0, 0.0] # "AI"
], dtype=torch.float32).unsqueeze(0) # 배치 차원 추가
print(f"입력 벡터:\n{x.squeeze(0)}")
# 간단한 선형 변환 (단위 행렬 사용)
W_q = torch.eye(d_model)
W_k = torch.eye(d_model)
W_v = torch.eye(d_model)
# Q, K, V 계산
Q = torch.matmul(x, W_q)
K = torch.matmul(x, W_k)
```

```
V = torch.matmul(x, W_v)

print(f"\nQuery 행렬:\n{Q.squeeze(0)}")

print(f"Key 행렬:\n{K.squeeze(0)}")

print(f"Value 행렬:\n{V.squeeze(0)}")

# 어텐션 계산

output, attention_weights = scaled_dot_product_attention(Q, K, V)

print(f"\n어텐션 가중치:\n{attention_weights.squeeze(0)}")

print(f"출력:\n{output.squeeze(0)}")

return output, attention_weights



# 실행

output, weights = simple_attention_example()
```

예제 2: 마스킹이 있는 어텐션

```
def  masked_attention_example():

"""

마스킹이 적용된 어텐션 예제 (디코더에서 사용)

"""

print("\n=== 마스킹 어텐션 예제 ===")

seq_len = 4

d_model = 8

# 샘플 입력
```

```
x = torch.randn(1, seq_len, d_model)

# 하삼각 마스크 생성 (미래 토큰을 보지 못하게)

mask = torch.tril(torch.ones(seq_len, seq_len))

print(f"마스크:\n{mask}")

# 어텐션 레이어

attention_layer = AttentionLayer(d_model)

# 마스킹 없는 어텐션

output_no_mask, weights_no_mask = attention_layer(x)

# 마스킹 있는 어텐션

output_masked, weights_masked = attention_layer(x, mask)

print(f"\n마스킹 없는 어텐션 가중치:\n{weights_no_mask.squeeze(0)}")

print(f"마스킹 있는 어텐션 가중치:\n{weights_masked.squeeze(0)}")

return weights_no_mask, weights_masked


# 실행

weights_no_mask, weights_masked = masked_attention_example()
```

예제 3: 어텐션 패턴 분석

```
def  analyze_attention_patterns():

"""

실제 문장에서 어텐션 패턴 분석

"""
```

```python
print("\n=== 어텐션 패턴 분석 ===")

# 예시 문장

sentence = "The quick brown fox jumps over the lazy dog"

tokens = sentence.split()

seq_len = len(tokens)

d_model = 64

# 어텐션 레이어

attention_layer = AttentionLayer(d_model)

# 랜덤 입력 (실제로는 단어 임베딩)

x = torch.randn(1, seq_len, d_model)

# 어텐션 계산

output, attention_weights = attention_layer(x)

attention_matrix = attention_weights.squeeze(0).detach()

# 각 단어별 가장 높은 어텐션을 받는 단어 찾기

print("각 단어가 가장 주의를 기울이는 단어:")

for i, token in  enumerate(tokens):

max_attention_idx = torch.argmax(attention_matrix[i])

max_attention_val = attention_matrix[i, max_attention_idx]

print(f"{token} -> {tokens[max_attention_idx]} (가중치: {max_attention_val:.4f})"

# 평균 어텐션 거리 계산

distances = []

for i in  range(seq_len):

for j in  range(seq_len):
```

```
weight = attention_matrix[i, j]

distance = abs(i - j)

distances.extend([distance] * int(weight * 100)) # 가중치에 비례하여 거리 추가

avg_distance = np.mean(distances)

print(f"\n평균 어텐션 거리: {avg_distance:.2f}")

return attention_matrix, tokens



# 실행

attention_matrix, tokens = analyze_attention_patterns()
```

핵심 개념 정리

1. 어텐션의 핵심 아이디어

- **선택적 집중**: 모든 입력에 동등하게 주의를 기울이지 않고, 중요한 부분에 집중
- **병렬 처리**: 모든 위치를 동시에 처리하여 계산 효율성 향상
- **장거리 의존성**: 멀리 떨어진 단어들 간의 관계도 직접적으로 모델링

2. 수학적 핵심

- **내적 유사도**: Query와 Key의 내적으로 관련성 측정
- **스케일링**: √d_k로 나누어 그래디언트 안정화
- **Softmax 정규화**: 확률 분포로 변환하여 해석 가능성 향상

3. 구현상 핵심

- **선형 변환**: 같은 입력에서 서로 다른 Q, K, V 생성

- **배치 처리**: 효율적인 행렬 연산으로 병렬 처리
- **마스킹**: 특정 위치에 대한 어텐션 차단

4. 어텐션의 장점

1. **해석 가능성**: 어텐션 가중치를 통해 모델의 집중 부분 확인 가능
2. **효율성**: 병렬 처리로 RNN 대비 빠른 학습
3. **성능**: 장거리 의존성 모델링으로 더 나은 성능
4. **유연성**: 다양한 태스크에 적용 가능

5. 다음 단계

이제 어텐션 메커니즘의 기본을 이해했으니, 다음 주제들을 학습할 수 있습니다:

- 포지셔널 인코딩: 순서 정보를 어떻게 주입하는가
- 멀티헤드 어텐션: 여러 개의 어텐션 헤드 사용
- 인코더 아키텍처: 어텐션을 포함한 전체 인코더 구조

이해도 확인 질문:

1. Query, Key, Value의 역할을 각각 설명해보세요.
2. 스케일링 팩터 √d_k가 필요한 이유는 무엇인가요?
3. 셀프 어텐션에서 각 위치가 자기 자신에게 주의를 기울이는 것이 왜 의미가 있을까요?
4. 어텐션 가중치의 합이 1이 되는 이유와 그 의미는 무엇인가요?

실습 과제:

1. 위의 코드를 실행하여 어텐션 가중치를 시각화해보세요.
2. 다른 문장으로 어텐션 패턴을 분석해보세요.
3. 마스킹의 효과를 직접 확인해보세요.